

جامعة أم القرى
UMM AL-QURA UNIVERSITY



Hospital Appointment Management System

Course Name: Introduction to Database
Instructor Name: Dr. Ruba Aljamal
Semester: Second Semester – 1447 H

Prepared By:

Raghad Alnufie – 446001465
Jana Haron – 44611038
Reema Alsharif – 446001714
Layan Alshaikhi – 446010013



TEAM MEMBERS CONTRIBUTION	3
SECTION 1	4
CONCEPTUAL DESIGN (SCENARIO).....	5
LOGICAL DESIGN (ERD)	6
LOGICAL DESIGN (RELATIONAL SCHEMA)	7
SECTION 2	8
CREATE DATABASE	9
PATIENT TABLE	10
DOCTOR TABLE	11
APPOINTMENT TABLE	12
SECTION 3	13
BASIC SELECT QUERIES	14
AGGREGATE FUNCTIONS	15
JOIN QUERY.....	16
SUBQUERY	17
VIEW.....	18



Team Members Contribution

The project tasks were carefully divided among the team members to ensure equal participation and effective collaboration. Each member was responsible for a specific part of the project as described below.

Student Name	Responsibilities
Raghad Alnufie	Raghad designed the relational schema and worked on the Doctor table code by creating the table and inserting its sample data. She also implemented part of the SQL queries and prepared the related screenshots.
Jana Haron	Jana worked on the Patient table code by creating the table and inserting its sample data. She also implemented the Basic SELECT queries and prepared the screenshots related to the Patient table and query results.
Reema Alsharif	Reema coordinated the project workflow as the team leader. She wrote the project scenario, prepared the cover page, table of contents, and task distribution table, designed the ER diagram, implemented the Aggregate Functions queries, and reviewed the final report to improve its organization.
Layan Alshaikhi	Layan worked on the Appointment table code by creating the table and inserting its sample data. She also implemented the JOIN query, subquery, and view, and prepared the related screenshots.



Section 1



Conceptual Design (Scenario)

The hospital requires a database system to manage appointments between patients and doctors in an organized and efficient manner. The purpose of this system is to store and manage information about patients, doctors, and scheduled appointments while ensuring data accuracy and data integrity.

The system is designed to simplify appointment scheduling and maintain structured records. It helps the hospital avoid data redundancy and ensures that each appointment is properly linked to the correct patient and doctor.

The database includes three main entities: Patient, Doctor, and Appointment.

Each patient is identified by a unique Patient_ID and has a Name and Phone number. Each doctor is identified by a unique Doctor_ID and has a Name and Specialty.

Each appointment is identified by a unique Appt_ID and includes a Date and a Status, such as Scheduled, Completed, or Cancelled.

In this system, a patient can book multiple appointments over time. Similarly, a doctor can attend multiple appointments with different patients.

However, each appointment is associated with exactly one patient and exactly one doctor.

To represent these relationships, the Appointment entity includes references to both Patient and Doctor. This ensures that every appointment record clearly identifies the patient who booked it and the doctor who attends it.

This structured database design allows the hospital to manage appointments efficiently while maintaining accurate and consistent records.



Logical Design (ERD)

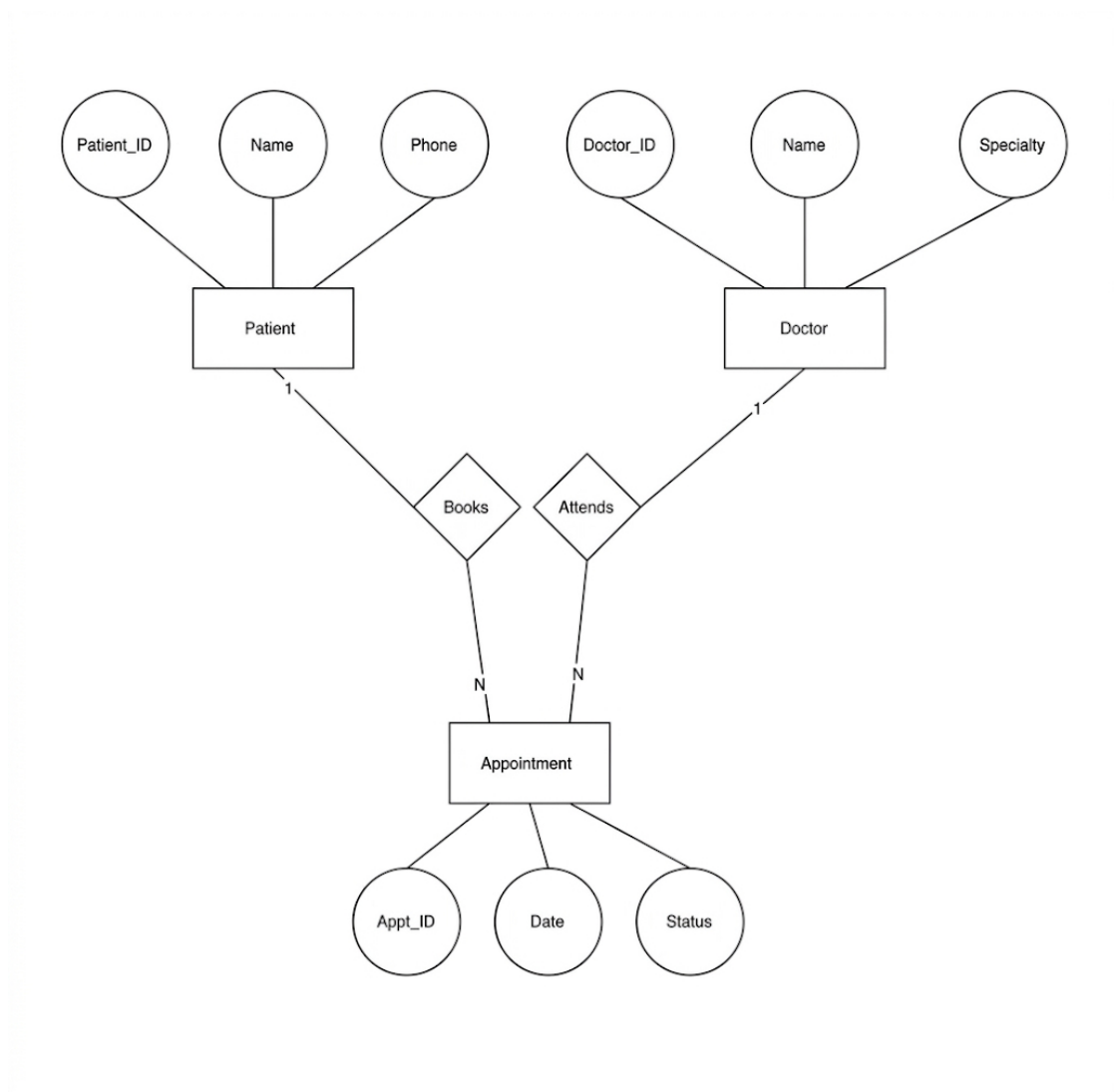


Figure 1: ER Diagram



Logical Design (Relational Schema)

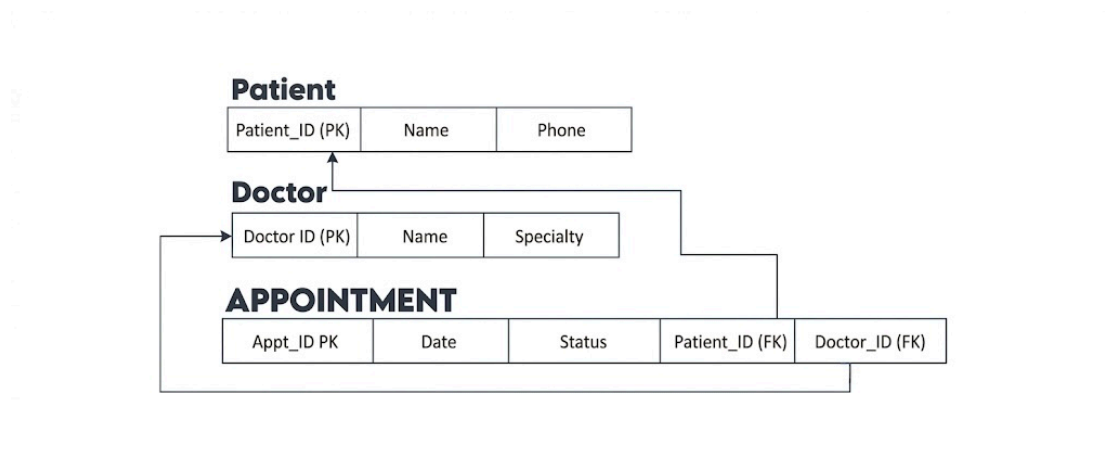


Figure 2: Relational Schema



Section 2



Create Database

The screenshot displays the MySQL Workbench interface for a local instance. The SQL editor contains the following commands:

```
1  
2  
3  
4  
5  
6 • CREATE DATABASE Hospital_Appointment_Management_System;  
7 • USE Hospital_Appointment_Management_System;  
8 • show databases;  
9  
10  
11  
12
```

The **Result Grid** shows the output of the `show databases;` command:

Database
Hospital_Appointment_Management_System
information_schema
mysql
performance_schema
sys

The **Action Output** pane at the bottom shows the execution log:

Action	Time	Action	Response	Duration / Fetch Time
✓ 13	14:24:00	USE Hospital_Appointment_Management_System	0 row(s) affected	0.00021 sec
✓ 14	14:24:16	USE Hospital_Appointment_Management_System	0 row(s) affected	0.00021 sec
✓ 15	15:02:40	show databases	5 row(s) returned	0.0020 sec / 0.00001...

Figure 3: Database Setup (CREATE, USE, SHOW)



Patient Table

Local instance 3306 - Warning - not supported

Administration Schemas Query 1

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Context Help Snippets

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```
1 CREATE DATABASE Hospital_Appointment_Management_System;
2 USE Hospital_Appointment_Management_System;
3
4 CREATE TABLE Patient (
5     Patient_ID INT PRIMARY KEY,
6     Name VARCHAR(100) NOT NULL,
7     Phone VARCHAR(15) NOT NULL UNIQUE
8 );
9
10 INSERT INTO Patient (Patient_ID, Name, Phone) VALUES
11 (1, 'Ahmed Ali', '0501234567'),
12 (2, 'Sara Mohammed', '0502345678'),
13 (3, 'Khalid Hassan', '0503456789'),
14 (4, 'Noura Salem', '0504567890'),
15 (5, 'Faisal Omar', '0505678901');
16
17 SELECT * FROM Patient;
18 DESCRIBE Patient;
```

Result Grid

Patient_ID	Name	Phone
1	Ahmed Ali	0501234567
2	Sara Mohammed	0502345678
3	Khalid Hassan	0503456789
4	Noura Salem	0504567890
5	Faisal Omar	0505678901

Object Info Session

No object selected

Patient 1

Apply Revert

Figure 4: Patient Table Data

Local instance 3306 - Warning - not supported

Administration Schemas Query 1

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Context Help Snippets

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

```
1 CREATE DATABASE Hospital_Appointment_Management_System;
2 USE Hospital_Appointment_Management_System;
3
4 CREATE TABLE Patient (
5     Patient_ID INT PRIMARY KEY,
6     Name VARCHAR(100) NOT NULL,
7     Phone VARCHAR(15) NOT NULL UNIQUE
8 );
9
10 INSERT INTO Patient (Patient_ID, Name, Phone) VALUES
11 (1, 'Ahmed Ali', '0501234567'),
12 (2, 'Sara Mohammed', '0502345678'),
13 (3, 'Khalid Hassan', '0503456789'),
14 (4, 'Noura Salem', '0504567890'),
15 (5, 'Faisal Omar', '0505678901');
16
17 SELECT * FROM Patient;
18 DESCRIBE Patient;
```

Result Grid

Field	Type	Null	Key	Default	Extra
Patient_ID	int	NO	PRI		
Name	varchar(100)	NO			
Phone	varchar(15)	NO	UNI		

Object Info Session

No object selected

Result 2

Read Only

Figure 5: Patient Table Structure



Doctor Table

The screenshot shows the SQL Developer interface with a query window containing the following SQL code:

```
20
21 CREATE TABLE Doctor (
22   Doctor_ID INT PRIMARY KEY,
23   Name VARCHAR(100) NOT NULL,
24   Specialty VARCHAR(100) NOT NULL
25 );
26 INSERT INTO Doctor (Doctor_ID, Name, Specialty) VALUES
27 (101, 'Dr. Huda Ibrahim', 'Cardiology'),
28 (102, 'Dr. Omar Saeed', 'Dermatology'),
29 (103, 'Dr. Lina Adel', 'Pediatrics'),
30 (104, 'Dr. Yousif Khalid', 'Orthopedics'),
31 (105, 'Dr. Mona Ahmed', 'Neurology');
32 SELECT * FROM Doctor;
33 DESCRIBE Doctor;
34
35
```

The result grid below the query shows the data inserted into the Doctor table:

Doctor_ID	Name	Specialty
101	Dr. Huda Ibrahim	Cardiology
102	Dr. Omar Saeed	Dermatology
103	Dr. Lina Adel	Pediatrics
104	Dr. Yousif Khalid	Orthopedics
105	Dr. Mona Ahmed	Neurology

Figure 6: Doctor Table Data

The screenshot shows the SQL Developer interface with the same SQL code as Figure 6. The result grid below the query shows the structure of the Doctor table:

Field	Type	Null	Key	Default	Extra
Doctor_ID	int	NO	PRI		
Name	varchar(100)	NO			
Specialty	varchar(100)	NO			

Figure 7: Doctor Table Structure



Appointment Table

Figure 8: Appointment Table Data

Figure 9: Appointment Table Structure



Section 3



Basic SELECT Queries

The screenshot shows the SQL Enterprise Manager interface. The left pane displays the 'Administration' tree with 'Server Status' selected. The main pane shows a query window with the following SQL code:

```
-- section 3
SELECT Name, Phone
FROM Patient
WHERE Patient_ID = 1;
```

The 'Result Grid' shows one row of data:

Name	Phone
Ahmed Ali	0501234567

The 'Action Output' pane shows the execution results:

Time	Action	Response	Duration / Fetch Time
13:39:36	SELECT Doctor_ID, AVG(Appointment_ID) AS avg_appointments FROM Appointment GRO...	5 row(s) returned	0.0044 sec / 0.0000...
13:43:13	SELECT Name, Specialty FROM Doctor WHERE Specialty = 'Cardiology' LIMIT 0, 1...	1 row(s) returned	0.0016 sec / 0.00000...
13:43:16	SELECT Name, Phone FROM Patient WHERE Patient_ID = 1 LIMIT 0, 1000	1 row(s) returned	0.0026 sec / 0.00000...

Figure 10: Basic SELECT Queries1

The screenshot shows the SQL Enterprise Manager interface. The left pane displays the 'Administration' tree with 'Server Status' selected. The main pane shows a query window with the following SQL code:

```
-- section 3
SELECT Name, Phone
FROM Patient
WHERE Patient_ID = 1;

SELECT Name, Specialty
FROM Doctor
WHERE Specialty = 'Cardiology';
```

The 'Result Grid' shows two rows of data:

Name	Specialty
Dr. Huda Ibrahim	Cardiology

The 'Action Output' pane shows the execution results:

Time	Action	Response	Duration / Fetch Time
13:43:13	SELECT Name, Specialty FROM Doctor WHERE Specialty = 'Cardiology' LIMIT 0, 1...	1 row(s) returned	0.0016 sec / 0.00000...
13:43:16	SELECT Name, Phone FROM Patient WHERE Patient_ID = 1 LIMIT 0, 1000	1 row(s) returned	0.0026 sec / 0.00000...
13:43:26	SELECT Name, Specialty FROM Doctor WHERE Specialty = 'Cardiology' LIMIT 0, 1...	1 row(s) returned	0.00029 sec / 0.0000...

Figure 11: Basic SELECT Queries2



Aggregate Functions

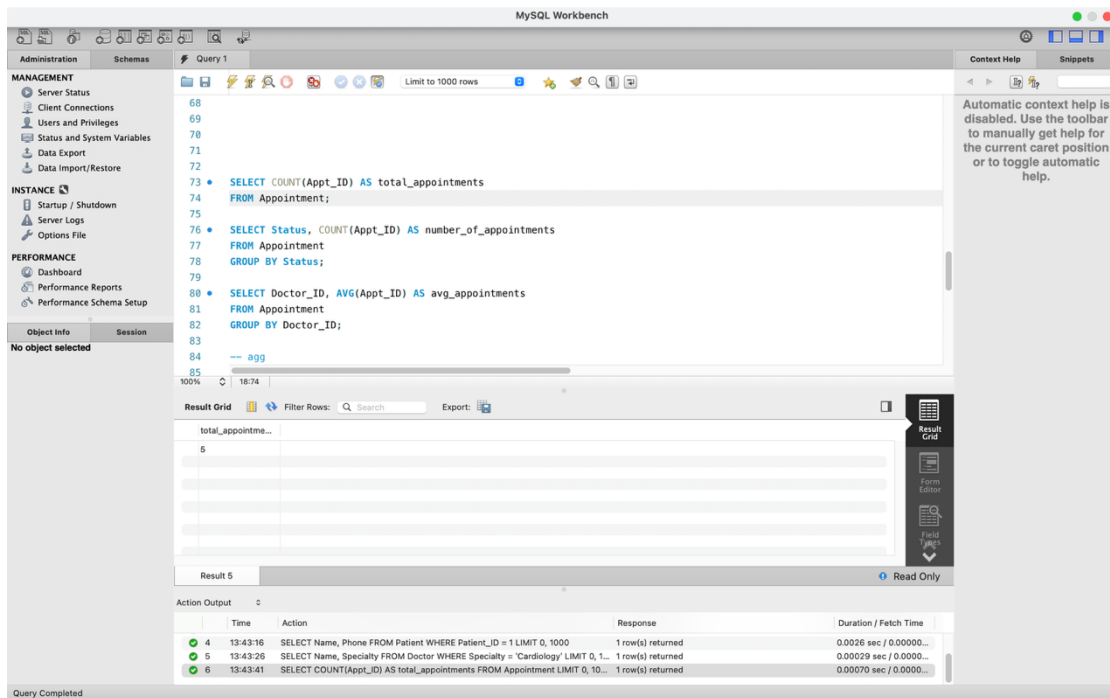


Figure 12: Aggregate Functions1

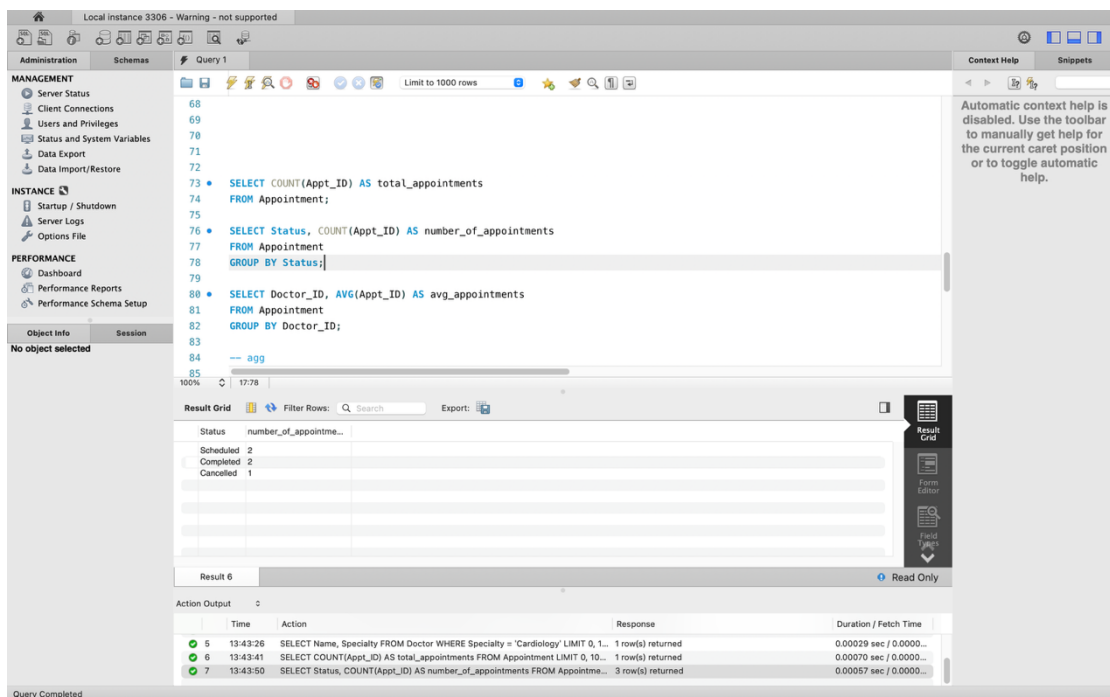


Figure 13: Aggregate Functions2



JOIN Query

The screenshot displays the SQL Enterprise Manager interface for a local instance. The query editor shows a SELECT statement with an INNER JOIN between the Appointment table (a) and the Patient table (p) on the Patient_ID column, and another INNER JOIN with the Doctor table (d) on the Doctor_ID column. The query is limited to 1000 rows.

```
83
84
85
86 SELECT
87     p.Name AS patient_name,
88     d.Name AS doctor_name,
89     a.Date,
90     a.Status
91 FROM Appointment a
92 INNER JOIN Patient p ON a.Patient_ID = p.Patient_ID
93 INNER JOIN Doctor d ON a.Doctor_ID = d.Doctor_ID
```

The Results pane shows a table with 5 rows and 4 columns: patient_name, doctor_name, Date, and Status.

patient_name	doctor_name	Date	Status
Ahmed Ali	Dr. Huda Ibrahim	2029-04-01	Scheduled
Sara Mohammed	Dr. Omar Saeed	2029-04-08	Completed
Khalid Hassan	Dr. Lisa Adel	2029-04-05	Cancelled
Noura Salem	Dr. Yousif Khalid	2029-04-07	Scheduled
Faisal Omar	Dr. Mona Ahmed	2029-04-09	Completed

The Action Output pane shows the execution of the query, indicating that 5 rows were returned.

Time	Action	Response	Duration / Fetch Time
13:43:50	SELECT Status, COUNT(App_ID) AS number_of_appointments FROM Appointment	3 row(s) returned	0.00057 sec / 0.0000...
13:44:01	SELECT Doctor_ID, AVG(App_ID) AS avg_appointments FROM Appointment GRO...	5 row(s) returned	0.00051 sec / 0.0000...
13:44:42	SELECT p.Name AS patient_name, d.Name AS doctor_name, a.Date, a...	5 row(s) returned	0.00038 sec / 0.0000...

Figure 14: JOIN Query



Subquery

Question: Which doctors have scheduled appointments?

The screenshot shows the SQL Enterprise Manager interface. The Query Editor contains the following SQL query:

```
SELECT Name, Specialty
FROM Doctor
WHERE Doctor_ID IN (
  SELECT Doctor_ID
  FROM Appointment
  WHERE Status = 'Scheduled'
);
```

The Results pane displays the following data:

Name	Specialty
Dr. Huda Ibrahim	Cardiology
Dr. Yousef Khalid	Orthopedics

The Action Output pane shows the execution details of the query:

Time	Action	Response	Duration / Fetch Time
13:44:01	SELECT Doctor_ID, AVG(Appointment_ID) AS avg_appointments FROM Appointment GRO...	5 row(s) returned	0.00051 sec / 0.0000...
13:44:42	SELECT p.Name AS patient_name, d.Name AS doctor_name, a.Date, a...	5 row(s) returned	0.00038 sec / 0.000...
13:44:55	SELECT Name, Specialty FROM Doctor WHERE Doctor_ID IN (SELECT Doctor_...	2 row(s) returned	0.00061 sec / 0.0000...

Figure 15: Subquery



View

Local instance 3306 - Warning - not supported

Administration Schemas Query 1

MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore

INSTANCE
Startup / Shutdown
Server Logs
Options File

PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup

Object Info Session
No object selected

Query 1

```
110  
111  
112  
113  
114 • CREATE VIEW Appointment_Report AS  
115 SELECT  
116     a.Appt_ID,  
117     p.Name AS patient_name,  
118     d.Name AS doctor_name,  
119     a.Date,  
120     a.Status  
121 FROM Appointment a  
122 INNER JOIN Patient p ON a.Patient_ID = p.Patient_ID  
123 INNER JOIN Doctor d ON a.Doctor_ID = d.Doctor_ID;  
124  
125 • SELECT * FROM Appointment_Report;  
126
```

Limit to 1000 rows

Context Help Snippets

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Result Grid

Appt_ID	patient_name	doctor_name	Date	Status
1001	Ahmed Ali	Dr. Huda Ibrahim	2026-04-01	Scheduled
1002	Sara Mohammed	Dr. Omar Saeed	2026-04-03	Completed
1003	Khalid Hassan	Dr. Lina Adel	2026-04-05	Cancelled
1004	Noura Salem	Dr. Youssif Khalid	2026-04-07	Scheduled
1005	Faisal Omar	Dr. Mona Ahmed	2026-04-09	Completed

Appointment_Report 10 Read Only

Action Output

Time	Action	Response	Duration / Fetch Time
10 13:44:55	SELECT Name, Specialty FROM Doctor WHERE Doctor_ID IN (SELECT Doctor_ID FROM Appointment_Report)	2 row(s) returned	0.00061 sec / 0.0000...
11 13:45:13	CREATE VIEW Appointment_Report AS SELECT a.Appt_ID, p.Name AS patient_name, d.Name AS doctor_name, a.Date, a.Status FROM Appointment a INNER JOIN Patient p ON a.Patient_ID = p.Patient_ID INNER JOIN Doctor d ON a.Doctor_ID = d.Doctor_ID	Error Code: 1050. Table 'Appointment_Report' already exists.	0.00096 sec
12 13:45:17	SELECT * FROM Appointment_Report LIMIT 0, 1000	5 row(s) returned	0.00069 sec / 0.0000...

Query Completed

Figure 16: View Result